

Computing Admissible Strings I

Normal forms, CKY, pushdown automata, and parsing

Cem Bozsahin

LING487 · Fall 2026

After the midterm, the question changes

description $\longrightarrow$ method

Question

Given a grammar and a string, how do we systematically determine admissibility?

- Normal forms standardize rule shape.
- CKY records every licensed constituent span.
- Pushdown automata turn recursive structure into state-and-stack computation.

MOLR §§4.5–4.6; 5.1–5.3

Where we are

1. Normal forms
2. CKY
3. Pushdown automata
4. Top-down and bottom-up parsing

Normal form trades surface variety for a uniform method

Chomsky normal form

Every rule is $A \rightarrow BC$ or $A \rightarrow a$, with a controlled exception for $S \rightarrow \epsilon$.

Greibach normal form

Every nonempty rule begins with a terminal: $A \rightarrow aA_1 \cdots A_n$.

Take-away

Equivalent normal-form grammars preserve stringsets but need not preserve the original structural analysis.

Binarization creates the spans CKY needs

$$A \rightarrow BCD$$

can be replaced by

$$A \rightarrow BX, \quad X \rightarrow CD.$$

- Every nonlexical combination now joins exactly two adjacent spans.
- The added category X is an artifact of the method unless independently interpreted.

Guided analysis

Binarize $S \rightarrow NP \ V \ NP$ in two ways. Do the resulting trees encode the same grouping?

MOLR §4.5

Where we are

1. Normal forms
2. CKY
3. Pushdown automata
4. Top-down and bottom-up parsing

CKY turns parsing into a triangular record of spans

For a string $w_1 \dots w_n$, cell $[i, j]$ contains every category that derives $w_i \dots w_j$.

- ① Put lexical categories on the diagonal.
- ② For each longer span, try every split point.
- ③ Combine $B \in [i, k]$ and $C \in [k + 1, j]$ when $A \rightarrow BC$.
- ④ Accept if $S \in [1, n]$.

A four-word CKY chart records all binary combinations

[1, 4]			
[1, 3]	[2, 4]		
[1, 2]	[2, 3]	[3, 4]	
w_1	w_2	w_3	w_4

Question

How many split points must be checked for cell $[1, 4]$?

Ambiguity survives as multiple derivations in one cell

For $n + n * n$, a grammar with

$$E \rightarrow E + E \mid E * E \mid n$$

admits both

$$(n + n) * n \quad \text{and} \quad n + (n * n).$$

- The top cell can contain one category with two backpointer structures.
- Recognition answers “admitted?”; parsing also recovers how.

Guided analysis

Which added grammar distinction would encode conventional precedence?

Where we are

1. Normal forms
2. CKY
3. Pushdown automata
4. Top-down and bottom-up parsing

A pushdown automaton adds unbounded stack memory

$$M = (Q, \Sigma, \Gamma, \Delta, s, F)$$

- Q : states; Σ : input alphabet; Γ : stack alphabet.
- Δ : transition relation; s : start; F : accepting states.
- A configuration $[q, \alpha, \Gamma^*]$ records state, unread input, and stack.

MOLR §5.1

The stack preserves a dependency after its trigger disappears

To recognize $\{a^n b^n \mid n \geq 0\}$:

- ① Push one marker for every a .
- ② On the first b , switch to popping.
- ③ Pop one marker for every b .
- ④ Accept exactly when input and stack are exhausted appropriately.

Take-away

The stack is a method for carrying the coordinated count described by the grammar.

Where we are

1. Normal forms
2. CKY
3. Pushdown automata
4. Top-down and bottom-up parsing

Top-down parsing predicts; bottom-up parsing recognizes

Top-down

Begin with S ; expand expected categories; match terminals against input.

Bottom-up

Begin with input; shift terminals; reduce recognized right-hand sides to categories.

Question

Which direction risks pursuing expectations that the input never supports? Which risks building irrelevant fragments?

MOLR §§5.2–5.3

Left recursion is a methodological problem, not an illicit structure

$$A \rightarrow A\alpha \mid \beta$$

can loop in a naive top-down parser before consuming input. A standard rewrite is

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \epsilon.$$

- The rewrite preserves the stringset under suitable conditions.
- It changes the derivational shape used by the parser.

Guided analysis

Eliminate left recursion from $E \rightarrow E + n \mid n$.

Third hour: compute one grammar three ways

Use $S \rightarrow AB$, $A \rightarrow aA \mid a$, $B \rightarrow bB \mid b$.

- 1 Normalize it for CKY.
- 2 Fill a CKY chart for *aabb*.
- 3 Sketch the stack idea for a top-down PDA.
- 4 Describe what a bottom-up parser shifts and reduces.
- 5 Identify which changes are methodological rather than linguistic.

Week 9 synthesis

- 1 Normal forms enable uniform methods but may alter visible structure.
- 2 CKY computes every licensed constituent span.
- 3 Recognition and parsing answer different questions.
- 4 A PDA carries context-free dependency in a stack.
- 5 Top-down and bottom-up methods expose different search problems.

Question

Quiz 7 asks you to carry out or interpret one small computational step.